

Physics Hand

1.2.6

Generated by Doxygen 1.9.5

1 Namespace Index	1
1.1 Package List	1
2 Hierarchical Index	3
2.1 Class Hierarchy	3
3 Class Index	5
3.1 Class List	5
4 Namespace Documentation	7
4.1 PhysicsHand Namespace Reference	7
4.2 PhysicsHand.Collisions Namespace Reference	7
4.3 PhysicsHand.Editor Namespace Reference	7
4.4 PhysicsHand.Events Namespace Reference	8
4.5 PhysicsHand.GrabSystem Namespace Reference	8
4.6 PhysicsHand.GrabSystem.Sliders Namespace Reference	8
4.7 PhysicsHand.Helpers Namespace Reference	8
4.8 PhysicsHand.RigidbodyHand Namespace Reference	8
4.9 PhysicsHand.Structures Namespace Reference	9
4.10 PhysicsHand.UI Namespace Reference	9
4.11 PhysicsHand.UI.Events Namespace Reference	9
5 Class Documentation	11
5.1 PhysicsHand.GrabSystem.Sliders.AnimatedGrabSlider Class Reference	11
5.1.1 Detailed Description	12
5.1.2 Member Function Documentation	12
5.1.2.1 OnSlideGrabbed()	12
5.1.2.2 OnSlideReleased()	13
5.2 PhysicsHand.RigidbodyHand.BoxCastState Class Reference	13
5.3 PhysicsHand.GrabbableHandBend.Entry Class Reference	13
5.3.1 Member Function Documentation	14
5.3.1.1 CheckBendConditions()	14
5.4 PhysicsHand.GrabbableHandPose.Entry Class Reference	14
5.4.1 Member Function Documentation	14
5.4.1.1 CheckPoseConditions()	14
5.5 PhysicsHand.GrabSystem.Sliders.GrabSliderInvoker.Entry Class Reference	15
5.6 PhysicsHand.Structures.FloatMinMax Struct Reference	15
5.7 PhysicsHand.GrabbableHandBend Class Reference	16
5.7.1 Detailed Description	16
5.8 PhysicsHand.GrabbableHandPose Class Reference	17
5.8.1 Detailed Description	17
5.9 PhysicsHand.GrabSystem.Sliders.GrabSliderInvoker Class Reference	18
5.9.1 Detailed Description	18
5.9.2 Member Function Documentation	18

5.9.2.1 OnSlideGrabbed()	18
5.9.2.2 OnSlideReleased()	19
5.10 PhysicsHand.Collisions.HandsIntercollisionManager Class Reference	19
5.10.1 Detailed Description	20
5.10.2 Member Function Documentation	20
5.10.2.1 ForceIgnoreIntercollisions()	20
5.10.2.2 IgnoreIntercollisionFor()	20
5.10.2.3 SetIgnoreIntercollisions()	21
5.10.2.4 UnignoreIntercollisionIn()	21
5.11 PhysicsHand.Rigidbody.InheritRigidbodyVelocity Class Reference	21
5.11.1 Detailed Description	22
5.12 PhysicsHand.GrabSystem.ReleaseNocollideGrace Class Reference	22
5.12.1 Detailed Description	22
5.13 PhysicsHand.RigidbodyHand Class Reference	23
5.13.1 Detailed Description	25
5.13.2 Member Function Documentation	25
5.13.2.1 PlayHapticImpulse()	25
5.13.2.2 SetAllFingerBendIfNotGrabbing()	25
5.13.2.3 SetDynamicGrabBoxEnabled()	26
5.13.2.4 SetForceBendOnGrabEnabled()	26
5.13.2.5 SetGrabFingerBend()	26
5.13.2.6 SetRestingFingerBend()	27
5.13.2.7 SetUnbendObstructedIfNotGrabbing()	27
5.13.3 Member Data Documentation	27
5.13.3.1 closedCastInfo	27
5.13.3.2 openedCastInfo	28
5.14 PhysicsHand.UI.RigidbodyHandUIInputModule Class Reference	28
5.14.1 Detailed Description	28
5.14.2 Member Function Documentation	29
5.14.2.1 GetEventData()	29
5.14.2.2 Press()	29
5.14.2.3 RegisterUIPointer()	29
5.14.2.4 Release()	30
5.14.2.5 UnregisterUIPointer()	30
5.15 PhysicsHand.UI.RigidbodyHandUIPointer Class Reference	30
5.15.1 Detailed Description	32
5.15.2 Member Function Documentation	32
5.15.2.1 SetAutoPressUIDistance()	32
5.15.2.2 SetMaxUIDistance()	33
5.15.2.3 TakeoverWorldCanvases()	33
5.16 PhysicsHand.Events.RigidbodyHandUnityEvent Class Reference	33
5.16.1 Detailed Description	33

5.17 PhysicsHand.RigidbodyStabilizer Class Reference	34
5.17.1 Detailed Description	34
5.18 PhysicsHand.RigidbodyStabilizer.Target Class Reference	35
5.19 PhysicsHand.UI.Events.UIPointerUnityEvent Class Reference	35
5.19.1 Detailed Description	35
5.20 PhysicsHand.UI.RigidbodyHandUIPointer.Vector3UnityEvent Class Reference	35
5.20.1 Detailed Description	36
5.21 PhysicsHand.Structures.ZeroOneFloatMinMax Struct Reference	36
Index	37

Chapter 1

Namespace Index

1.1 Package List

Here are the packages with brief descriptions (if available):

PhysicsHand	7
PhysicsHand.Collisions	7
PhysicsHand.Editor	7
PhysicsHand.Events	8
PhysicsHand.GrabSystem	8
PhysicsHand.GrabSystem.Sliders	8
PhysicsHand.Helpers	8
PhysicsHand.Rigidbodyies	8
PhysicsHand.Structures	9
PhysicsHand.UI	9
PhysicsHand.UI.Events	9

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

BaseInputModule	
PhysicsHand.UI.RigidbodyHandUIInputModule	28
PhysicsHand.RigidbodyHand.BoxCastState	13
PhysicsHand.GrabbableHandBend.Entry	13
PhysicsHand.GrabbableHandPose.Entry	14
PhysicsHand.GrabSystem.Sliders.GrabSliderInvoker.Entry	15
PhysicsHand.Structures.FloatMinMax	15
MonoBehaviour	
PhysicsHand.Collisions.HandsIntercollisionManager	19
PhysicsHand.GrabSystem.ReleaseNocollideGrace	22
PhysicsHand.GrabSystem.Sliders.AnimatedGrabSlider	11
PhysicsHand.GrabSystem.Sliders.GrabSliderInvoker	18
PhysicsHand.GrabbableHandBend	16
PhysicsHand.GrabbableHandPose	17
PhysicsHand.Rigidbody.InheritRigidbodyVelocity	21
PhysicsHand.Rigidbody.RigidbodyStabilizer	34
PhysicsHand.RigidbodyHand	23
PhysicsHand.UI.RigidbodyHandUIPointer	30
PhysicsHand.Rigidbody.RigidbodyStabilizer.Target	35
UnityEvent	
PhysicsHand.Events.RigidbodyHandUnityEvent	33
PhysicsHand.UI.Events.UIPointerUnityEvent	35
PhysicsHand.UI.RigidbodyHandUIPointer.Vector3UnityEvent	35
PhysicsHand.Structures.ZeroOneFloatMinMax	36

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

PhysicsHand.GrabSystem.Sliders.AnimatedGrabSlider	
A component that allows a grabber to control a something signed distance driven sliding animation that sets normalized time in an animation clip using the controller's signed distance along a direction vector from the slide grab starting point to determine what point of the animation to play, the animation will be played in reverse to simulate the slide 'returning to start'	11
PhysicsHand.RigidbodyHand.BoxCastState	13
PhysicsHand.GrabbableHandBend.Entry	13
PhysicsHand.GrabbableHandPose.Entry	14
PhysicsHand.GrabSystem.Sliders.GrabSliderInvoker.Entry	15
PhysicsHand.Structures.FloatMinMax	15
PhysicsHand.GrabbableHandBend	
A component that allows you to specify the all-finger bend value for a hand that grabs a GrabbableObject on the same GameObject as this. This component is attached to the same GameObject as a GrabbableObject to control the hands finger bend when it is grabbing it . . .	16
PhysicsHand.GrabbableHandPose	
A component that allows you to specify the pose for a hand that grabs a GrabbableObject on the same GameObject as this. This component is attached to the same GameObject as a GrabbableObject to control the hands pose when it is grabbing it	17
PhysicsHand.GrabSystem.Sliders.GrabSliderInvoker	
A component that allows event(s) to be invoked when certain 'grab slider' conditions are met .	18
PhysicsHand.Collisions.HandsIntercollisionManager	
A component that manages intercollisions between RigidbodyHand components. By default this component makes a hand ignore intercollisions with a hand that is grabbing something while 'ignoreWhileGrabbing' is true.	19
PhysicsHand.RigidbodyBodies.InheritRigidbodyVelocity	
A component that can be attached to the same GameObject as a Rigidbody that allows you to specify another Rigidbody to inherit velocity from	21
PhysicsHand.GrabSystem.ReleaseNocollideGrace	
A component that allows release grace time to be overridden by attaching this component to the same GameObject as a GrabbableObject.	22
PhysicsHand.RigidbodyHand	
A Rigidbody based physics hand.	23
PhysicsHand.UI.RigidbodyHandUIInputModule	
An input module that managements UI inputs for RigidbodyHandUIPointers	28

PhysicsHand.UI.RigidbodyHandUIPointer	
A simple component that is attached to the same GameObject as a RigidbodyHand which enables UI interaction using the hand system. NOTE: You may only have 1 of these components enabled in a scene at any given time unless you manually configure the 'worldCamera' field of the relevant Canvases.	30
PhysicsHand.Events.RigidbodyHandUnityEvent	
Arg0: The RigidbodyHand associated with the event.	33
PhysicsHand.RigidbodyStabilizer	
A component that can be attached to the same GameObject as a Rigidbody component that visually stabilizes the Rigidbody by modifying the position pre-and-post render. This is useful for preventing jitter when Rigidbody s are jointed together	34
PhysicsHand.RigidbodyStabilizer.Target	35
PhysicsHand.UI.Events.UIPointerUnityEvent	
Arg0: Vector3 - The world space position the pointer hit. Arg1: GameObject - The GameObject the pointer hit.	35
PhysicsHand.UI.RigidbodyHandUIPointer.Vector3UnityEvent	
Arg0: Vector3 - the Vector3 passed to the event.	35
PhysicsHand.Structures.ZeroOneFloatMinMax	36

Chapter 4

Namespace Documentation

4.1 PhysicsHand Namespace Reference

Classes

- class [GrabbableHandBend](#)
A component that allows you to specify the all-finger bend value for a hand that grabs a GrabbableObject on the same GameObject as this. This component is attached to the same GameObject as a GrabbableObject to control the hands finger bend when it is grabbing it.
- class [GrabbableHandPose](#)
A component that allows you to specify the pose for a hand that grabs a GrabbableObject on the same GameObject as this. This component is attached to the same GameObject as a GrabbableObject to control the hands pose when it is grabbing it.
- class [RigidbodyHand](#)
A Rigidbody based physics hand.

4.2 PhysicsHand.Collisions Namespace Reference

Classes

- class [HandsIntercollisionManager](#)
A component that manages intercollisions between [RigidbodyHand](#) components. By default this component makes a hand ignore intercollisions with a hand that is grabbing something while 'ignoreWhileGrabbing' is true.

4.3 PhysicsHand.Editor Namespace Reference

Classes

- class [PhysicsHandEditorManager](#)
A static class that provides some useful functionality for keeping everything calculated and up-to-date while working in the editor.

4.4 PhysicsHand.Events Namespace Reference

Classes

- class [RigidbodyHandUnityEvent](#)
Arg0: The [RigidbodyHand](#) associated with the event.

4.5 PhysicsHand.GrabSystem Namespace Reference

Classes

- class **GrabUtilities**
Grab extension utility methods for the '[GrabSystem](#)' for Physics Hand.
- class [ReleaseNocollideGrace](#)
A component that allows release grace time to be overridden by attaching this component to the same GameObject as a GrabbableObject.

4.6 PhysicsHand.GrabSystem.Sliders Namespace Reference

Classes

- class [AnimatedGrabSlider](#)
A component that allows a grabber to control a something signed distance driven sliding animation that sets normalized time in an animation clip using the controller's signed distance along a direction vector from the slide grab starting point to determine what point of the animation to play, the animation will be played in reverse to simulate the slide 'returning to start'.
- class [GrabSliderInvoker](#)
A component that allows event(s) to be invoked when certain 'grab slider' conditions are met.

4.7 PhysicsHand.Helpers Namespace Reference

Classes

- class **FlagsHelper**
A component that helps check flags.

4.8 PhysicsHand.Rigidbodybodies Namespace Reference

Classes

- class [InheritRigidbodyVelocity](#)
A component that can be attached to the same GameObject as a Rigidbody that allows you to specify another Rigidbody to inherit velocity from.
- class [RigidbodyStabilizer](#)
A component that can be attached to the same GameObject as a Rigidbody component that visually stabilizes the Rigidbody by modifying the position pre-and-post render. This is useful for preventing jitter when [Rigidbodybodies](#) are jointed together.

4.9 PhysicsHand.Structures Namespace Reference

Classes

- struct [FloatMinMax](#)
- struct [ZeroOneFloatMinMax](#)

4.10 PhysicsHand.UI Namespace Reference

Classes

- class [RigidbodyHandUIInputModule](#)
An input module that managements [UI](#) inputs for [RigidbodyHandUIPointers](#).
- class [RigidbodyHandUIPointer](#)
A simple component that is attached to the same [GameObject](#) as a [RigidbodyHand](#) which enables [UI](#) interaction using the hand system. NOTE: You may only have 1 of these components enabled in a scene at any given time unless you manually configure the 'worldCamera' field of the relevant [Canvases](#).

4.11 PhysicsHand.UI.Events Namespace Reference

Classes

- class [UIPointerUnityEvent](#)
Arg0: [Vector3](#) - The world space position the pointer hit. Arg1: [GameObject](#) - The [GameObject](#) the pointer hit.

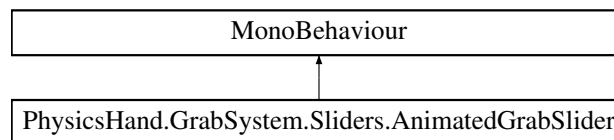
Chapter 5

Class Documentation

5.1 PhysicsHand.GrabSystem.Sliders.AnimatedGrabSlider Class Reference

A component that allows a grabber to control a something signed distance driven sliding animation that sets normalized time in an animation clip using the controller's signed distance along a direction vector from the slide grab starting point to determine what point of the animation to play, the animation will be played in reverse to simulate the slide 'returning to start'.

Inheritance diagram for PhysicsHand.GrabSystem.Sliders.AnimatedGrabSlider:



Public Member Functions

- void **SetAnimationNormalizedTime** (float pTime)
*A public method that allows the 'animation normalized time', the time in animation scaled from 0-> 1, to be set directly.
EXAMPLE USAGE: This is used in scenarios where for example sometihng overrides the slide's behaviour, such as firing the gun which would want to instantly make this component stop animating the slide by setting the animation normalized time to 0.*
- void **OnSlideGrabbed** (Grabber pGrabber, GrabbableObject pGrabbable)
A callback that is intended to be invoked by the slide's GrabbableObject component's 'Grabbed' event.
- void **OnSlideReleased** (Grabber pGrabber, GrabbableObject pGrabbable)
A callback that is intended to be invoked by the slide's GrabbableObjects component's 'Released' event..

Public Attributes

- float **maxSlideDistance**
The distance the controller must be along the
- float **slideReturnSpeed**
The speed the slide returns at once released in normalized time per second
- Vector3 **slideDirection**
The direction the slide should be
- string **slideAnimation** = ""
The name of the slide animation.
- int **animationLayer** = -1
The layer the slideAnimation is in.
- Animator **animator**
A reference to the animator this component references.

Properties

- float **AnimationNormalizedTime** [get, protected set]
The normalized time the animation state is being played at. (If m_GrabHandFollowing is null the animation state is not being played.)
- Transform **RelativeTo** [get]
Returns the Transform that directions are considered relative to.

5.1.1 Detailed Description

A component that allows a grabber to control a something signed distance driven sliding animation that sets normalized time in an animation clip using the controller's signed distance along a direction vector from the slide grab starting point to determine what point of the animation to play, the animation will be played in reverse to simulate the slide 'returning to start'.

WARNING: This component assumes that the slide can only be operated with one hand and WILL cause problems if two hands can grab the slide. Author: Mathew Aloisio

5.1.2 Member Function Documentation

5.1.2.1 OnSlideGrabbed()

```
void PhysicsHand.GrabSystem.Sliders.AnimatedGrabSlider.OnSlideGrabbed (
    Grabber pGrabber,
    GrabbableObject pGrabbable )
```

A callback that is intended to be invoked by the slide's GrabbableObject component's 'Grabbed' event.

Parameters

<i>pGrabber</i>	
<i>pGrabbable</i>	

5.1.2.2 OnSlideReleased()

```
void PhysicsHand.GrabSystem.Sliders.AnimatedGrabSlider.OnSlideReleased (
    Grabber pGrabber,
    GrabbableObject pGrabbable )
```

A callback that is intended to be invoked by the slide's GrabbableObjects component's 'Released' event..

Parameters

<i>pGrabber</i>	
<i>pGrabbable</i>	

The documentation for this class was generated from the following file:

- AnimatedGrabSlider.cs

5.2 PhysicsHand.RigidbodyHand.BoxCastState Class Reference

Public Attributes

- Vector3 **originPosition**
The local space position of box cast data
- RayGrabber.BoxCastData **boxCastData**
The box cast data to use for this state.

The documentation for this class was generated from the following file:

- RigidbodyHand.cs

5.3 PhysicsHand.GrabbableHandBend.Entry Class Reference

Public Member Functions

- bool [CheckBendConditions](#) (Grabber pGrabber, GrabbableObject pGrabbable)
A public method that checks if the bend conditions are met for this entry when pGrabber is grabbing pGrabbable (this frame).

Public Attributes

- float **fingerBend**
The finger bend value to apply to all fingers in a KinematicHand when this entry becomes active.

5.3.1 Member Function Documentation

5.3.1.1 CheckBendConditions()

```
bool PhysicsHand.GrabbableHandBend.Entry.CheckBendConditions (
    Grabber pGrabber,
    GrabbableObject pGrabbable )
```

A public method that checks if the bend conditions are met for this entry when pGrabber is grabbing pGrabbable (this frame).

Parameters

<i>pGrabber</i>	
<i>pGrabbable</i>	

Returns

true if the pose conditions for this entry are met, otherwise false.

The documentation for this class was generated from the following file:

- GrabbableHandBend.cs

5.4 PhysicsHand.GrabbableHandPose.Entry Class Reference

Public Member Functions

- bool [CheckPoseConditions](#) (Grabber pGrabber, GrabbableObject pGrabbable)
A public method that checks if the pose conditions are met for this entry when pGrabber is grabbing pGrabbable (this frame).

Public Attributes

- string **grabPose**
The name of the grab pose to use as it appears in the relevant HandPoser component.

5.4.1 Member Function Documentation

5.4.1.1 CheckPoseConditions()

```
bool PhysicsHand.GrabbableHandPose.Entry.CheckPoseConditions (
    Grabber pGrabber,
    GrabbableObject pGrabbable )
```

A public method that checks if the pose conditions are met for this entry when pGrabber is grabbing pGrabbable (this frame).

Parameters

<i>pGrabber</i>	
<i>pGrabbable</i>	

Returns

true if the pose conditions for this entry are met, otherwise false.

The documentation for this class was generated from the following file:

- GrabbableHandPose.cs

5.5 PhysicsHand.GrabSystem.Sliders.GrabSliderInvoker.Entry Class Reference

Public Attributes

- float **slideDistance**
The distance the controller must be along the
- Vector3 **slideDirection**
The direction the controller must be moved along to
- FloatMath.Comparator **comparator**
The comparator to use to check the signed distance against pullDistance.
- UnityEvent **EntryUsed**
An event that is invoked when the entry conditions were first met since last being non

The documentation for this class was generated from the following file:

- GrabSliderInvoker.cs

5.6 PhysicsHand.Structures.FloatMinMax Struct Reference

Public Attributes

- float **minimum**
The minimum value.
- float **maximum**
The maximum value.

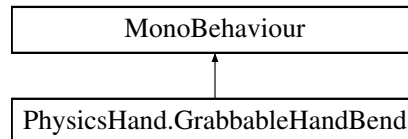
The documentation for this struct was generated from the following file:

- FloatMinMax.cs

5.7 PhysicsHand.GrabbableHandBend Class Reference

A component that allows you to specify the all-finger bend value for a hand that grabs a GrabbableObject on the same GameObject as this. This component is attached to the same GameObject as a GrabbableObject to control the hands finger bend when it is grabbing it.

Inheritance diagram for PhysicsHand.GrabbableHandBend:



Classes

- class [Entry](#)

Public Attributes

- bool **unbendFingers**
Should this component automatically unbend the fingers after the grabbable object is released? Recommended to keep this off if you use other scripts that control finger bend like for example Valve Index/Oculus Finger Tracking.
- [Entry\[\]](#) **grabBendEntries**
An array of GrabbableHandBend.Entrys that contain rules that decide which bend entry is used.

Properties

- GrabbableObject **Grabbable** [get]
Returns the GrabbableObject reference associated with this component.

5.7.1 Detailed Description

A component that allows you to specify the all-finger bend value for a hand that grabs a GrabbableObject on the same GameObject as this. This component is attached to the same GameObject as a GrabbableObject to control the hands finger bend when it is grabbing it.

If you want more precise control of finger bend consider using the HandPose component in conjunction wiht the [GrabbableHandPose](#) component instead.

TODO: Make some conditions that give the user control over which pose entry is used. (COPY FROM [GrabbableHandPose](#) when done TODODODOTODO TODO todo)

NOTE: This will only apply a pose to Grabbers that contain a KinematicHand component. Author: Mathew Aloisio

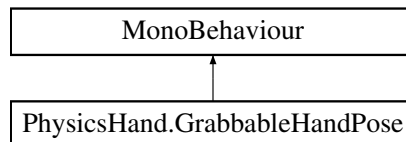
The documentation for this class was generated from the following file:

- GrabbableHandBend.cs

5.8 PhysicsHand.GrabbableHandPose Class Reference

A component that allows you to specify the pose for a hand that grabs a GrabbableObject on the same GameObject as this. This component is attached to the same GameObject as a GrabbableObject to control the hands pose when it is grabbing it.

Inheritance diagram for PhysicsHand.GrabbableHandPose:



Classes

- class [Entry](#)

Public Attributes

- [Entry\[\]](#) **grabPoseEntries**
An array of GrabbableHandPose.Entrys that contain rules that decide which grab pose is used.

Properties

- GrabbableObject **Grabbable** [get]
Returns the GrabbableObject reference associated with this component.

5.8.1 Detailed Description

A component that allows you to specify the pose for a hand that grabs a GrabbableObject on the same GameObject as this. This component is attached to the same GameObject as a GrabbableObject to control the hands pose when it is grabbing it.

TODO: Make some conditions that give the user control over which pose entry is used.

NOTE: This will only apply a pose to Grabbers that contain a HandPoser component. Author: Mathew Aloisio

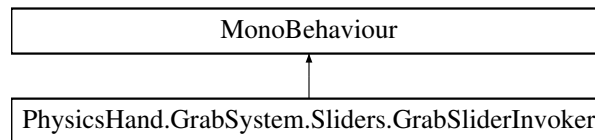
The documentation for this class was generated from the following file:

- GrabbableHandPose.cs

5.9 PhysicsHand.GrabSystem.Sliders.GrabSliderInvoker Class Reference

A component that allows event(s) to be invoked when certain 'grab slider' conditions are met.

Inheritance diagram for PhysicsHand.GrabSystem.Sliders.GrabSliderInvoker:



Classes

- class [Entry](#)

Public Member Functions

- void **RebuildEntries** ()
Rebuilds any internal data that is built using the 'entries' array.
- void **OnSlideGrabbed** (Grabber pGrabber, GrabbableObject pGrabbable)
A callback that is intended to be invoked by the slide's GrabbableObject component's 'Grabbed' event.
- void **OnSlideReleased** (Grabber pGrabber, GrabbableObject pGrabbable)
A callback that is intended to be invoked by the slide's GrabbableObjects component's 'Released' event..

Public Attributes

- [Entry\[\]](#) **entries**
An array of Entries and conditions that can be fired by this invoker.

Properties

- Transform **RelativeTransform** [get]
A reference to the Transform that pull directions are considered relative to.

5.9.1 Detailed Description

A component that allows event(s) to be invoked when certain 'grab slider' conditions are met.

WARNING: This component assumes that the slide can only be operated with one hand and WILL cause problems if two hands can grab the slide. Author: Mathew Aloisio

5.9.2 Member Function Documentation

5.9.2.1 OnSlideGrabbed()

```
void PhysicsHand.GrabSystem.Sliders.GrabSliderInvoker.OnSlideGrabbed (
    Grabber pGrabber,
    GrabbableObject pGrabbable )
```

A callback that is intended to be invoked by the slide's GrabbableObject component's 'Grabbed' event.

Parameters

<i>pGrabber</i>	
<i>pGrabbable</i>	

5.9.2.2 OnSlideReleased()

```
void PhysicsHand.GrabSystem.Sliders.GrabSliderInvoker.OnSlideReleased (
    Grabber pGrabber,
    GrabbableObject pGrabbable )
```

A callback that is intended to be invoked by the slide's GrabbableObjects component's 'Released' event..

Parameters

<i>pGrabber</i>	
<i>pGrabbable</i>	

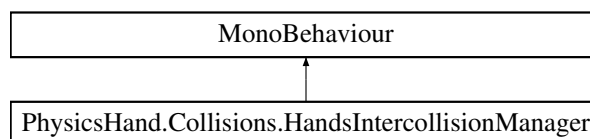
The documentation for this class was generated from the following file:

- GrabSliderInvoker.cs

5.10 PhysicsHand.Collisions.HandsIntercollisionManager Class Reference

A component that manages intercollisions between [RigidbodyHand](#) components. By default this component makes a hand ignore intercollisions with a hand that is grabbing something while 'ignoreWhileGrabbing' is true.

Inheritance diagram for PhysicsHand.Collisions.HandsIntercollisionManager:

**Public Member Functions**

- void [IgnoreIntercollisionFor](#) (float pSeconds)
 Ignores intercollisions between the hands for the given number of seconds. Will be ignored if intercollisions are already ignored due to a grab.
- void [UnignoreIntercollisionIn](#) (float pSeconds)
 Unignores intercollisions between the hands after the given number of seconds. Will be ignored if intercollisions are ignored due to a grab.
- void [SetIgnoreIntercollisions](#) (bool pIgnore)
 Sets the intercollision ignore state between hands registered with this components if nothing is overriding it. Will not overwrite if intercollisions are already ignored due to a grab.
- void [ForceIgnoreIntercollisions](#) (bool pIgnore)
 Forces the intercollision ignore state between hands registered with this components regardless of grab state, etc.

Public Attributes

- [RigidbodyHand\[\]](#) **hands**
An array of [RigidbodyHand](#) components whose inter
- bool **ignoreWhileGrabbing** = true
Should intercollisions be ignored while grabbing?
- float **releaseGrace** = 0.4f
The number of seconds hands remain no

Properties

- bool **IgnoringDueToGrab** [get]
Tracks whether or not this component is currently ignoring intercollisions due to a grab.

5.10.1 Detailed Description

A component that manages intercollisions between [RigidbodyHand](#) components. By default this component makes a hand ignore intercollisions with a hand that is grabbing something while 'ignoreWhileGrabbing' is true.

Author: Mathew Aloisio

5.10.2 Member Function Documentation

5.10.2.1 ForceIgnoreIntercollisions()

```
void PhysicsHand.Collisions.HandsIntercollisionManager.ForceIgnoreIntercollisions (
    bool pIgnore )
```

Forces the intercollision ignore state between hands registered with this components regardless of grab state, etc.

Parameters

<i>pIgnore</i>	Should intercollisions be ignored?
----------------	------------------------------------

5.10.2.2 IgnoreIntercollisionFor()

```
void PhysicsHand.Collisions.HandsIntercollisionManager.IgnoreIntercollisionFor (
    float pSeconds )
```

Ignores intercollisions between the hands for the given number of seconds. Will be ignored if intercollisions are already ignored due to a grab.

Parameters

<i>pSeconds</i>	
-----------------	--

5.10.2.3 SetIgnoreIntercollisions()

```
void PhysicsHand.Collisions.HandsIntercollisionManager.SetIgnoreIntercollisions (
    bool pIgnore )
```

Sets the intercollision ignore state between hands registered with this components if nothing is overriding it. Will not overwrite if intercollisions are already ignored due to a grab.

Parameters

<i>pIgnore</i>	Should intercollisions be ignored?
----------------	------------------------------------

5.10.2.4 UnignoreIntercollisionIn()

```
void PhysicsHand.Collisions.HandsIntercollisionManager.UnignoreIntercollisionIn (
    float pSeconds )
```

Unignores intercollisions between the hands after the given number of seconds. Will be ignored if intercollisions are ignored due to a grab.

Parameters

<i>pSeconds</i>	
-----------------	--

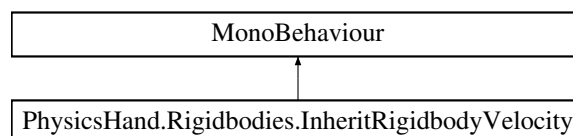
The documentation for this class was generated from the following file:

- HandsIntercollisionManager.cs

5.11 PhysicsHand.Rigidbody.InheritRigidbodyVelocity Class Reference

A component that can be attached to the same GameObject as a Rigidbody that allows you to specify another Rigidbody to inherit velocity from.

Inheritance diagram for PhysicsHand.Rigidbody.InheritRigidbodyVelocity:



Public Attributes

- [Rigidbody](#) **inheritFrom**
The Rigidbody to inherit velocity from.

Properties

- Rigidbody **Rigidbody** [get]
A reference to the Rigidbody associated with this component.

5.11.1 Detailed Description

A component that can be attached to the same GameObject as a Rigidbody that allows you to specify another Rigidbody to inherit velocity from.

NOTE: This component may not handle large sudden velocity changes of 'inheritFrom' well.

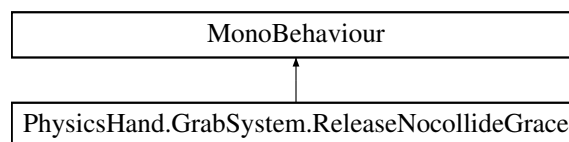
The documentation for this class was generated from the following file:

- InheritRigidbodyVelocity.cs

5.12 PhysicsHand.GrabSystem.ReleaseNocollideGrace Class Reference

A component that allows release grace time to be overridden by attaching this component to the same GameObject as a GrabbableObject.

Inheritance diagram for PhysicsHand.GrabSystem.ReleaseNocollideGrace:



Public Attributes

- float **releaseCollisionGrace**
The overridden release collision grace time in seconds for the relevant GrabbableObject.

5.12.1 Detailed Description

A component that allows release grace time to be overridden by attaching this component to the same GameObject as a GrabbableObject.

Author: Mathew Aloisio

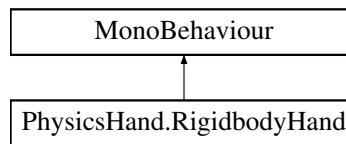
The documentation for this class was generated from the following file:

- ReleaseNocollideGrace.cs

5.13 PhysicsHand.RigidbodyHand Class Reference

A Rigidbody based physics hand.

Inheritance diagram for PhysicsHand.RigidbodyHand:



Classes

- class [BoxCastState](#)

Public Member Functions

- void **TeleportToFollow** ()
Instantly teleports the hand (and anything it is grabbing) to the ForceFollower.follow Transform. This only deals with position, not with rotation.
- void **RebuildDynamicGrabBox** ()
Rebuilds the dynamic grab box for the RayGrabber component associated with this hand.
- void **RebuildCollisionIgnorer** ()
Rebuilds the collision ignorer using the Collider components found in this hands Rigidbody.
- void [PlayHapticImpulse](#) (float pDuration, float pAmplitude, float pFrequency)
Plays a haptic impulse using the 'hapticsDevice' associated with the hand (if one exists).
- void **StopHaptics** ()
Stops any haptics being palyed by the 'hapticsDevice' associated with the hand (if there is one).
- void **SetAllFingerBendToGrabTarget** ()
Sets all finger bend values for the hand to 'grabFingerBend'.
- void [SetAllFingerBendIfNotGrabbing](#) (float pBend)
Sets all finger bend values for the hand if not grabbing.
- void **RestAllFingers** ()
Sets all finger bend values for the hand to 'restingFingerBend'.
- void **RestAllFingersIfNotGrabbing** ()
Sets all finger bend values for the hand to 'restingFingerBend' if not grabbing.
- void [SetUnbendObstructedIfNotGrabbing](#) (bool pUnbendObstructed)
Sets the value of 'unbend obstructed' for the relevant kinematic hand globally if not grabbing.
- void [SetDynamicGrabBoxEnabled](#) (bool pEnabled)
Sets the 'dynamicGrabBox' field of this component. Useful for use with Unity [Editor](#) events.
- void [SetForceBendOnGrabEnabled](#) (bool pEnabled)
Sets the 'forceBendOnGrab' field of this component. Useful for use with Unity [Editor](#) events.
- void [SetGrabFingerBend](#) (float pBend)
Sets the 'grabFingerBend' field of this component. Useful for use with Unity [Editor](#) events.
- void [SetRestingFingerBend](#) (float pBend)
Sets the 'defaultFingerBend' field of this component. Useful for use with Unity [Editor](#) events.

Public Attributes

- bool **leftHand**
Is this hand the left hand?
- float **grabFingerBend** = 1f
The target finger bend factor for the hand while grabbing.
- float **restingFingerBend** = 0.1f
The default
- Animator **animator**
- float **unbendObstructedThreshold** = 1f
The average finger bend value
- [ZeroOneFloatMinMax](#) **bendGrabRange** = new [ZeroOneFloatMinMax](#)() { minimum = 0.01f, maximum = 0.85f }
The valid average finger bend range for the hand outside of which the continuous grab routine is interrupted. Note that these limits only affect continuous grab attempts
- float **releaseCollisionGrace** = 0f
The number of seconds of grace time before collisions are restored between the grabbable object and the hand.
- float **minGraceVelocity** = 0.1f
Only apply release collision grace if the average hand velocity's square magnitude is
- float **bendDisableDelay** = 0.8f
After this many seconds the fingers will be
- bool **forceBendOnGrab** = true
Should the finger bend for all fingers be set to
- bool **dynamicGrabBox** = true
Should the box cast volume interpolate from closedCastInfo to openedCastInfo as KinematicHand.AverageFinger↔Bend moves from
- AnimationCurve **dynamicGrabCurve** = AnimationCurve.Linear(0f, 0f, 1f, 1f)
A curve that can be used to scale the rate at which an increasing average finger bend moves the cast info from
- [BoxCastState](#) **openedCastInfo**
The box cast state information to use when the average bend of the hands fingers is
- [BoxCastState](#) **closedCastInfo**
The box cast state information to use when the average bend of the hands fingers is
- bool **blockPoserInGrab** = true
Should the poser
- IHapticsDevice **hapticsDevice**
- UnityEvent **Started**
An event that is invoked at the end of this component

Properties

- RayGrabber **Grabber** [get]
A reference to the RayGrabber that is responsible for grabbing and throwing with this hand.
- Rigidbody **Rigidbody** [get]
A reference to the Rigidbody component that is associated with this hand.
- [RigidbodyStabilizer](#) **Stabilizer** [get]
A reference to the RigidbodyStabilizer component that is associated with this hand.
- KinematicHand **KinematicHand** [get]
A reference to the KinematicHand that visually represents the physics hand.
- CollisionIgnorer **CollisionIgnorer** [get, set]
A reference to the CollisionIgnorer associated with this component
- CollisionTracker **CollisionTracker** [get]

- A reference to the CollisionTracker used by this component.*
- ForceFollowTransform **ForceFollower** [get]

A reference to the ForceFollowTransform component used by this component to position and orientate the hand.
- Vector3 **AverageVelocity** [get]

Returns Grabber.velocityTracker.AverageVelocity if Grabber.velocityTracker is non-null, otherwise returns Rigidbody.velocity.
- bool **IsLeft** [get]

Returns true if this is a left hand, otherwise false.
- bool **IsRight** [get]

Returns true if this is a right hand, otherwise false.
- int **SkipMaintainOffsetFrames** [get, set]

A simple integer that can be set to a number to skip any grabbable object 'Maintain Offset' updates for this many FixedUpdate() frames.

5.13.1 Detailed Description

A Rigidbody based physics hand.

Author: Mathew Aloisio

5.13.2 Member Function Documentation

5.13.2.1 PlayHapticImpulse()

```
void PhysicsHand.RigidbodyHand.PlayHapticImpulse (
    float pDuration,
    float pAmplitude,
    float pFrequency )
```

Plays a haptic impulse using the 'hapticsDevice' associated with the hand (if one exists).

Parameters

<i>pDuration</i>	
<i>pAmplitude</i>	
<i>pFrequency</i>	

5.13.2.2 SetAllFingerBendIfNotGrabbing()

```
void PhysicsHand.RigidbodyHand.SetAllFingerBendIfNotGrabbing (
    float pBend )
```

Sets all finger bend values for the hand if not grabbing.

Parameters

<i>pBend</i>	
--------------	--

5.13.2.3 SetDynamicGrabBoxEnabled()

```
void PhysicsHand.RigidbodyHand.SetDynamicGrabBoxEnabled (
    bool pEnabled )
```

Sets the 'dynamicGrabBox' field of this component. Useful for use with Unity [Editor](#) events.

Parameters

<i>pEnabled</i>	
-----------------	--

5.13.2.4 SetForceBendOnGrabEnabled()

```
void PhysicsHand.RigidbodyHand.SetForceBendOnGrabEnabled (
    bool pEnabled )
```

Sets the 'forceBendOnGrab' field of this component. Useful for use with Unity [Editor](#) events.

Parameters

<i>pEnabled</i>	
-----------------	--

5.13.2.5 SetGrabFingerBend()

```
void PhysicsHand.RigidbodyHand.SetGrabFingerBend (
    float pBend )
```

Sets the 'grabFingerBend' field of this component. Useful for use with Unity [Editor](#) events.

Parameters

<i>pBend</i>	
--------------	--

5.13.2.6 SetRestingFingerBend()

```
void PhysicsHand.RigidbodyHand.SetRestingFingerBend (
    float pBend )
```

Sets the 'defaultFingerBend' field of this component. Useful for use with Unity [Editor](#) events.

Parameters

<i>pBend</i>	
--------------	--

5.13.2.7 SetUnbendObstructedIfNotGrabbing()

```
void PhysicsHand.RigidbodyHand.SetUnbendObstructedIfNotGrabbing (
    bool pUnbendObstructed )
```

Sets the value of 'unbend obstructed' for the relevant kinematic hand globally if not grabbing.

Parameters

<i>pUnbendObstructed</i>	
--------------------------	--

5.13.3 Member Data Documentation

5.13.3.1 closedCastInfo

[BoxCastState](#) PhysicsHand.RigidbodyHand.closedCastInfo

Initial value:

```
= new BoxCastState()
{
    originPosition = new Vector3(0.0236f, -0.0157f, 0.1204f), boxCastData = new
    RayGrabber.BoxCastData() { boxHeight = 0.02f, boxWidth = 0.06f, boxLength = 0.13f }
}
```

The box cast state information to use when the average bend of the hands fingers is

5.13.3.2 openedCastInfo

`BoxCastState` `PhysicsHand.RigidbodyHand.openedCastInfo`

Initial value:

```
= new BoxCastState()
{
    originPosition = new Vector3(0.0236f, -0.0164f, 0.1204f),
    boxCastData = new RayGrabber.BoxCastData() { boxHeight = 0.018f, boxWidth = 0.06f, boxLength =
0.04f }
}
```

The box cast state information to use when the average bend of the hands fingers is

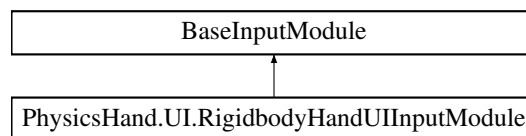
The documentation for this class was generated from the following file:

- `RigidbodyHand.cs`

5.14 PhysicsHand.UI.RigidbodyHandUIInputModule Class Reference

An input module that managements [UI](#) inputs for `RigidbodyHandUIPointers`.

Inheritance diagram for `PhysicsHand.UI.RigidbodyHandUIInputModule`:



Public Member Functions

- `int RegisterUIPointer (RigidbodyHandUIPointer pPointer)`
Registers a [RigidbodyHandUIPointer](#), `pPointer`, with the input module.
- `bool UnregisterUIPointer (RigidbodyHandUIPointer pPointer)`
Unregisters a [RigidbodyHandUIPointer](#), `pPointer`, with the input module.
- `void RebuildEventData ()`
Rebuilds the underlying event data array.
- `PointerEventData GetEventData (int pIndex)`
Returns the `PointerEventData` in the specified index, `pIndex`, from the underlying event data array.
- `void Press (int pIndex)`
Invokes a press event for the pointer in the given index.
- `void Release (int pIndex)`
Invokes a release event for the pointer in the given index.
- `override void Process ()`

5.14.1 Detailed Description

An input module that managements [UI](#) inputs for `RigidbodyHandUIPointers`.

Author: Mathew Aloisio

5.14.2 Member Function Documentation

5.14.2.1 GetEventData()

```
PointerEventData PhysicsHand.UI.RigidbodyHandUIInputModule.GetEventData (
    int pIndex )
```

Returns the PointerEventData in the specified index, *pIndex*, from the underlying event data array.

Parameters

<i>pIndex</i>	
---------------	--

Returns

the PointerEventData in the specified index, *pIndex*, from the underlying event data array.

5.14.2.2 Press()

```
void PhysicsHand.UI.RigidbodyHandUIInputModule.Press (
    int pIndex )
```

Invokes a press event for the pointer in the given index.

Parameters

<i>pIndex</i>	
---------------	--

5.14.2.3 RegisterUIPointer()

```
int PhysicsHand.UI.RigidbodyHandUIInputModule.RegisterUIPointer (
    RigidbodyHandUIPointer pPointer )
```

Registers a [RigidbodyHandUIPointer](#), *pPointer*, with the input module.

Parameters

<i>pPointer</i>	
-----------------	--

Returns

an integer representing the index of the newly added [UI](#) pointer in the internal pointers array.

5.14.2.4 Release()

```
void PhysicsHand.UI.RigidbodyHandUIInputModule.Release (
    int pIndex )
```

Invokes a release event for the pointer in the given index.

Parameters

<i>pIndex</i>	
---------------	--

5.14.2.5 UnregisterUIPointer()

```
bool PhysicsHand.UI.RigidbodyHandUIInputModule.UnregisterUIPointer (
    RigidbodyHandUIPointer pPointer )
```

Unregisters a [RigidbodyHandUIPointer](#), pPointer, with the input module.

Parameters

<i>pPointer</i>	
-----------------	--

Returns

true if the [UI](#) pointer was removed, otherwise false.

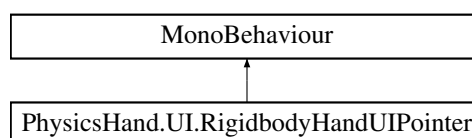
The documentation for this class was generated from the following file:

- RigidbodyHandUIInputModule.cs

5.15 PhysicsHand.UI.RigidbodyHandUIPointer Class Reference

A simple component that is attached to the same GameObject as a [RigidbodyHand](#) which enables [UI](#) interaction using the hand system. NOTE: You may only have 1 of these components enabled in a scene at any given time unless you manually configure the 'worldCamera' field of the relevant Canvases.

Inheritance diagram for PhysicsHand.UI.RigidbodyHandUIPointer:



Classes

- class [Vector3UnityEvent](#)
Arg0: Vector3 - the Vector3 passed to the event.

Public Member Functions

- void **Press** ()
Simulates a press using the [UI](#) pointer.
- void **Release** ()
Simulates a release using the [UI](#) pointer.
- void [TakeoverWorldCanvases](#) (bool pOverride=false)
Finds all Canvas components in the scene and takes control of them.
- void [SetAutoPressUIDistance](#) (float pDistance)
Sets the 'autoPressUIDistance' field of this component. Useful for use with Unity editor events.
- void [SetMaxUIDistance](#) (float pDistance)
A public method that sets the 'maxUIDistance' field of this component. Useful for use with Unity editor events.

Public Attributes

- float **autoPressUIDistance** = 0.05f
The raycast distance at which the [UI](#) pointer will automatically
- float **maxUIDistance** = 1f
The maximum distance this [UI](#) pointer can point.
- LayerMask **uiLayers**
A layer mask of valid [UI](#) layers.
- bool **rayHitTriggers** = true
Should the hovering/selection raycasts hit triggers?
- bool **takeoverCanvases** = true
Automatically takeover all world space canvases when enabled? If yes this pointers [UI](#) pointer camera will become the
- Transform **hitMarker**
- Vector3 **hitMarkerOffset**
The offset of the hit marker in local space of the hit marker.
- [UIPointerUnityEvent](#) **SelectStarted**
Invoked whenever the [UI](#) pointer begins selecting an object.
lnArg
- [UIPointerUnityEvent](#) **SelectStopped**
Invoked whenever the [UI](#) pointer stops selecting an object.
lnArg
- [UIPointerUnityEvent](#) **HoverStarted**
Invoked whenever the [UI](#) pointer begins hovering over an object.
lnArg
- [UIPointerUnityEvent](#) **HoverStopped**
Invoked whenever the [UI](#) pointer stops hovering over an object.
lnArg
- UnityEvent **UpdateTarget**
Invoked at the end of every frame with a valid target out of auto
- UnityEvent **UpdateAutoPressTarget**
Invoked at the end of every frame with a valid target within auto
- UnityEvent **UpdateNoTarget**
Invoked at the end of every target
- [Vector3UnityEvent](#) **RayEndPosCalculated**
Invoked every frame that the ray length of the pointer is calculated.
lnArg

Properties

- [RigidbodyHand](#) **Hand** [get]
A reference to the [RigidbodyHand](#) this [UI](#) pointer is using.
- Transform **UIPointer** [get]
A reference to the [UI](#) pointer Transform associated with this component.
- int **PointerIndex** [get, set]
The index of this pointer in the relevant [RigidbodyHandUIInputModule](#).
- EventSystem **EventSystem** [get]
A reference to the EventSystem associated with this component.
- [RigidbodyHandUIInputModule](#) **InputModule** [get]
A reference to the input module that is managing this pointer.
- GameObject **Target** [get]
A reference to the current target of the [UI](#) pointer, otherwise null.
- RaycastHit **LastPointerHit** [get]
The last RaycastHit data from pointer cast.
- Camera **PointerRaycastCamera** [get]
A reference to the Camera used by this component to raycast. (Automatically created.)
- bool **IsHovering** [get]
Returns true if this pointer is currently hovering over a target, otherwise false.
- bool **IsAutoPressing** [get]
Returns true if this component is currently auto pressing, otherwise false.

5.15.1 Detailed Description

A simple component that is attached to the same GameObject as a [RigidbodyHand](#) which enables [UI](#) interaction using the hand system. NOTE: You may only have 1 of these components enabled in a scene at any given time unless you manually configure the 'worldCamera' field of the relevant Canvases.

Author: Mathew Aloisio

5.15.2 Member Function Documentation

5.15.2.1 SetAutoPressUIDistance()

```
void PhysicsHand.UI.RigidbodyHandUIPointer.SetAutoPressUIDistance (
    float pDistance )
```

Sets the 'autoPressUIDistance' field of this component. Useful for use with Unity editor events.

Parameters

<i>pDistance</i>	
------------------	--

5.15.2.2 SetMaxUIDistance()

```
void PhysicsHand.UI.RigidbodyHandUIPointer.SetMaxUIDistance (
    float pDistance )
```

A public method that sets the 'maxUIDistance' field of this component. Useful for use with Unity editor events.

Parameters

<i>pDistance</i>	
------------------	--

5.15.2.3 TakeoverWorldCanvases()

```
void PhysicsHand.UI.RigidbodyHandUIPointer.TakeoverWorldCanvases (
    bool pOverride = false )
```

Finds all Canvas components in the scene and takes control of them.

Parameters

<i>pOverride</i>	Should non-null canvas.worldCamera references be overridden? If true yes, otherwise false if you don't want them overridden. (Default: false)
------------------	---

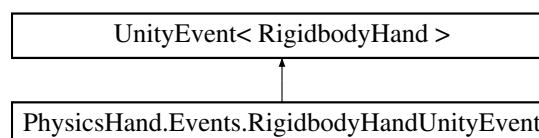
The documentation for this class was generated from the following file:

- RigidbodyHandUIPointer.cs

5.16 PhysicsHand.Events.RigidbodyHandUnityEvent Class Reference

Arg0: The [RigidbodyHand](#) associated with the event.

Inheritance diagram for PhysicsHand.Events.RigidbodyHandUnityEvent:



5.16.1 Detailed Description

Arg0: The [RigidbodyHand](#) associated with the event.

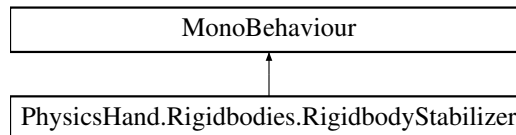
The documentation for this class was generated from the following file:

- RigidbodyHandUnityEvent.cs

5.17 PhysicsHand.RigidbodyStabilizer Class Reference

A component that can be attached to the same GameObject as a Rigidbody component that visually stabilizes the Rigidbody by modifying the position pre-and-post render. This is useful for preventing jitter when [Rigidbody](#)s are jointed together.

Inheritance diagram for PhysicsHand.RigidbodyStabilizer:



Classes

- class [Target](#)

Properties

- Rigidbody **Rigidbody** [get]
A reference to the Rigidbody associated with this component.
- bool **IsPreRenderAdjusted** [get]
Returns true while this component is in the 'PreRender' state before reaching 'PostRender'.
- Vector3 **PreRenderPosition** [get]
The pre-render position of the transform before being stabilized.
- Quaternion **PreRenderRotation** [get]
The pre-render rotation of the transform before being stabilized.
- [Target](#) **StabilizeTarget** [get, set]
The [RigidbodyStabilizer.Target](#) to place the stabilized transform at after storing pre-render position and rotation. (When null no stabilization occurs.)

5.17.1 Detailed Description

A component that can be attached to the same GameObject as a Rigidbody component that visually stabilizes the Rigidbody by modifying the position pre-and-post render. This is useful for preventing jitter when [Rigidbody](#)s are jointed together.

WARNING: This component may cause issues on pipelines other than the built-in render pipeline. It is not recommended to use this component on other render pipelines. Author: Mathew Aloisio

The documentation for this class was generated from the following file:

- RigidbodyStabilizer.cs

5.18 PhysicsHand.Rigidbody.RigidbodyStabilizer.Target Class Reference

Public Attributes

- Transform **transform**
The Transform of the target.

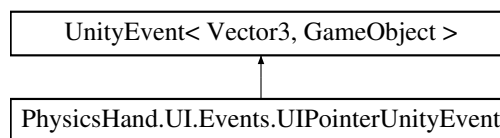
The documentation for this class was generated from the following file:

- RigidbodyStabilizer.cs

5.19 PhysicsHand.UI.Events.UIPointerUnityEvent Class Reference

Arg0: Vector3 - The world space position the pointer hit. Arg1: GameObject - The GameObject the pointer hit.

Inheritance diagram for PhysicsHand.UI.Events.UIPointerUnityEvent:



5.19.1 Detailed Description

Arg0: Vector3 - The world space position the pointer hit. Arg1: GameObject - The GameObject the pointer hit.

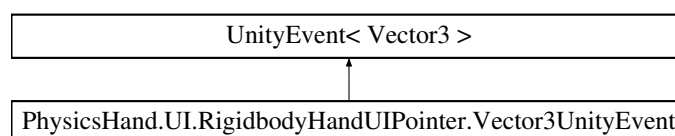
The documentation for this class was generated from the following file:

- UIPointerUnityEvent.cs

5.20 PhysicsHand.UI.RigidbodyHandUIPointer.Vector3UnityEvent Class Reference

Arg0: Vector3 - the Vector3 passed to the event.

Inheritance diagram for PhysicsHand.UI.RigidbodyHandUIPointer.Vector3UnityEvent:



5.20.1 Detailed Description

Arg0: Vector3 - the Vector3 passed to the event.

The documentation for this class was generated from the following file:

- RigidbodyHandUIPointer.cs

5.21 PhysicsHand.Structures.ZeroOneFloatMinMax Struct Reference

Public Attributes

- float **minimum**
The minimum value.
- float **maximum**
The maximum value.

The documentation for this struct was generated from the following file:

- ZeroOneFloatMinMax.cs

Index

CheckBendConditions
 PhysicsHand.GrabbableHandBend.Entry, 14

CheckPoseConditions
 PhysicsHand.GrabbableHandPose.Entry, 14

closedCastInfo
 PhysicsHand.RigidbodyHand, 27

ForceIgnoreIntercollisions
 PhysicsHand.Collisions.HandsIntercollisionManager, 20

GetEventData
 PhysicsHand.UI.RigidbodyHandUIInputModule, 29

IgnoreIntercollisionFor
 PhysicsHand.Collisions.HandsIntercollisionManager, 20

OnSlideGrabbed
 PhysicsHand.GrabSystem.Sliders.AnimatedGrabSlider, 12
 PhysicsHand.GrabSystem.Sliders.GrabSliderInvoker, 18

OnSlideReleased
 PhysicsHand.GrabSystem.Sliders.AnimatedGrabSlider, 13
 PhysicsHand.GrabSystem.Sliders.GrabSliderInvoker, 19

openedCastInfo
 PhysicsHand.RigidbodyHand, 27

PhysicsHand, 7

PhysicsHand.Collisions, 7

PhysicsHand.Collisions.HandsIntercollisionManager, 19

 ForceIgnoreIntercollisions, 20

 IgnoreIntercollisionFor, 20

 SetIgnoreIntercollisions, 21

 UnignoreIntercollisionIn, 21

PhysicsHand.Editor, 7

PhysicsHand.Events, 8

PhysicsHand.Events.RigidbodyHandUnityEvent, 33

PhysicsHand.GrabbableHandBend, 16

PhysicsHand.GrabbableHandBend.Entry, 13

 CheckBendConditions, 14

PhysicsHand.GrabbableHandPose, 17

PhysicsHand.GrabbableHandPose.Entry, 14

 CheckPoseConditions, 14

PhysicsHand.GrabSystem, 8

PhysicsHand.GrabSystem.ReleaseNocollideGrace, 22

PhysicsHand.GrabSystem.Sliders, 8

 PhysicsHand.GrabSystem.Sliders.AnimatedGrabSlider, 11

 OnSlideGrabbed, 12

 OnSlideReleased, 13

 PhysicsHand.GrabSystem.Sliders.GrabSliderInvoker, 18

 OnSlideGrabbed, 18

 OnSlideReleased, 19

 PhysicsHand.GrabSystem.Sliders.GrabSliderInvoker.Entry, 15

 PhysicsHand.Helpers, 8

 PhysicsHand.Rigidbody, 8

 PhysicsHand.Rigidbody.InheritRigidbodyVelocity, 21

 PhysicsHand.Rigidbody.RigidbodyStabilizer, 34

 PhysicsHand.Rigidbody.RigidbodyStabilizer.Target, 35

 PhysicsHand.RigidbodyHand, 23

 closedCastInfo, 27

 openedCastInfo, 27

 PlayHapticImpulse, 25

 SetAllFingerBendIfNotGrabbing, 25

 SetDynamicGrabBoxEnabled, 26

 SetForceBendOnGrabEnabled, 26

 SetGrabFingerBend, 26

 SetRestingFingerBend, 26

 SetUnbendObstructedIfNotGrabbing, 27

 PhysicsHand.RigidbodyHand.BoxCastState, 13

 PhysicsHand.Structures, 9

 PhysicsHand.Structures.FloatMinMax, 15

 PhysicsHand.Structures.ZeroOneFloatMinMax, 36

 PhysicsHand.UI, 9

 PhysicsHand.UI.Events, 9

 PhysicsHand.UI.Events.UIPointerUnityEvent, 35

 PhysicsHand.UI.RigidbodyHandUIInputModule, 28

 GetEventData, 29

 Press, 29

 RegisterUIPointer, 29

 Release, 30

 UnregisterUIPointer, 30

 PhysicsHand.UI.RigidbodyHandUIPointer, 30

 SetAutoPressUIDistance, 32

 SetMaxUIDistance, 32

 TakeoverWorldCanvases, 33

 PhysicsHand.UI.RigidbodyHandUIPointer.Vector3UnityEvent, 35

 PlayHapticImpulse

 PhysicsHand.RigidbodyHand, 25

 Press

 PhysicsHand.UI.RigidbodyHandUIInputModule, 29

 RegisterUIPointer

- PhysicsHand.UI.RigidbodyHandUIInputModule, [29](#)
- Release
 - PhysicsHand.UI.RigidbodyHandUIInputModule, [30](#)
- SetAllFingerBendIfNotGrabbing
 - PhysicsHand.RigidbodyHand, [25](#)
- SetAutoPressUIDistance
 - PhysicsHand.UI.RigidbodyHandUIPointer, [32](#)
- SetDynamicGrabBoxEnabled
 - PhysicsHand.RigidbodyHand, [26](#)
- SetForceBendOnGrabEnabled
 - PhysicsHand.RigidbodyHand, [26](#)
- SetGrabFingerBend
 - PhysicsHand.RigidbodyHand, [26](#)
- SetIgnoreIntercollisions
 - PhysicsHand.Collisions.HandsIntercollisionManager, [21](#)
- SetMaxUIDistance
 - PhysicsHand.UI.RigidbodyHandUIPointer, [32](#)
- SetRestingFingerBend
 - PhysicsHand.RigidbodyHand, [26](#)
- SetUnbendObstructedIfNotGrabbing
 - PhysicsHand.RigidbodyHand, [27](#)
- TakeoverWorldCanvases
 - PhysicsHand.UI.RigidbodyHandUIPointer, [33](#)
- UnignoreIntercollisionIn
 - PhysicsHand.Collisions.HandsIntercollisionManager, [21](#)
- UnregisterUIPointer
 - PhysicsHand.UI.RigidbodyHandUIInputModule, [30](#)